

Apprentissage statistique : modélisation décisionnelle et apprentissage profond (RCP209)

Réseaux convolutifs avancés

Nicolas Audebert

`nicolas.audebert@lecnam.net`

`http://cedric.cnam.fr/vertigo/Cours/ml2/`

Département Informatique
Conservatoire National des Arts & Métiers, Paris, France

1^{er} juin 2023

Plan du cours

1 Architectures résiduelles

2 Détection d'objet

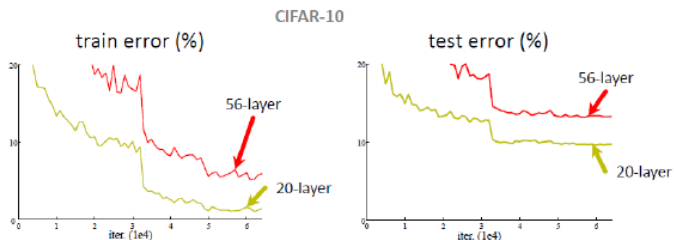
3 Segmentation d'images

Pourquoi les réseaux plus profonds apprennent moins bien ?

Deeper VGG: 56 Plain Network

Plain nets: stacking 3x3 conv layers

- 56-layer net has higher training error and test error than 20-layers net

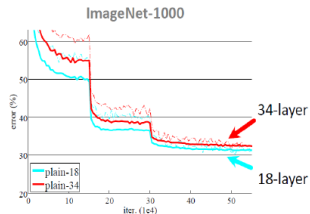
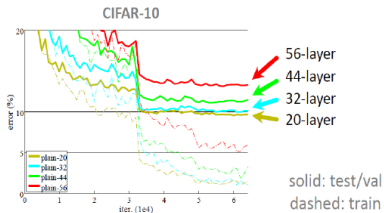


Pourquoi les réseaux plus profonds apprennent moins bien ?

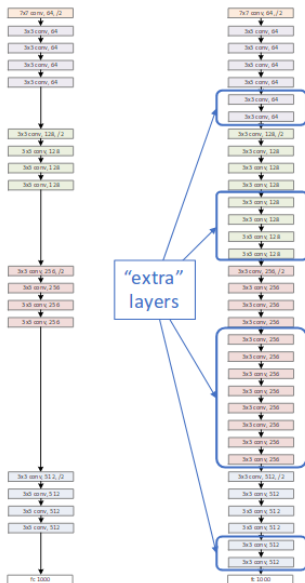
Deeper VGG:

“Overly deep” plain nets have higher training error

A general phenomenon, observed in many datasets



Problème : construction de modèles plus profond



- 18 couches (g.) contre 34 couches (d.)
- Modèle plus profond = plus expressif (l'ensemble des fonctions approchables est plus grand) $\Rightarrow$ devrait avoir une erreur d'apprentissage *plus faible*!
- Une expérience : construire un réseau plus profond à partir d'un réseau moins profond
 - Les couches "extra" sont initialisées comme étant l'identité
- $\Rightarrow$ le modèle plus profond est (au départ) équivalent au modèle moins profond
- Mais le modèle plus profond ne converge pas non plus vers une meilleure erreur d'apprentissage...
- $\Rightarrow$ problème d'optimisation : les solveurs basés gradient ne parviennent pas à trouver de meilleurs minima dans des modèles plus profonds!

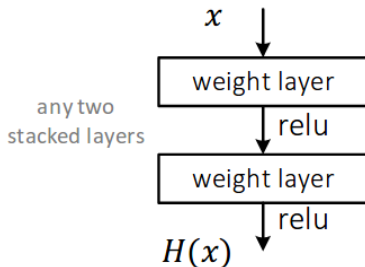
Identity mapping

Comment aider le solveur à apprendre des modèles plus profonds ?

- Intuition : puisque les modèles moyennement profonds donnent déjà des performances élevées, une partie des couches dans les modèles très profonds sera proche de l'identité ("*identity mapping*")
- En notant $H(x)$ la fonction de transfert modélisée par deux couches avec des paramètres :

$$H(x) = \sigma(\mathbf{W}_2(\sigma(\mathbf{W}_1 + 1)) + 2)$$

- Pour un empilement de plusieurs couches non-linéaires, il est difficile d'apprendre à modéliser la fonction identité !

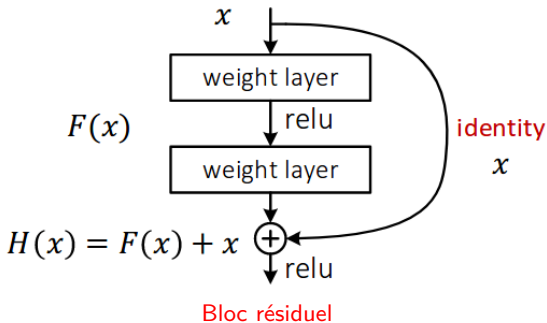


Blocs résiduels

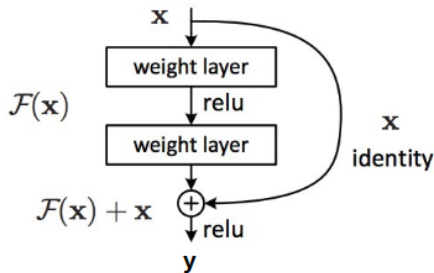
- Si $H(x)$ est la fonction “idéale”, et $F(x)$ la fonction apprise par un bloc de deux couches, alors plutôt que d'essayer de rapprocher F de H , on peut apprendre seulement le “résidu” :

$$H(x) = F(x) + x \quad \text{ou encore} \quad F(x) = H(x) - x$$

- Si la fonction optimale est l'identité ($H(x) = x$), alors il suffit de mettre tous les poids du bloc à 0.
- Si la fonction optimale est proche de l'identité, alors il sera possible d'apprendre des petites déviations à l'identité.



Bloc résiduel et flot du gradient



$$y = x + F(x)$$

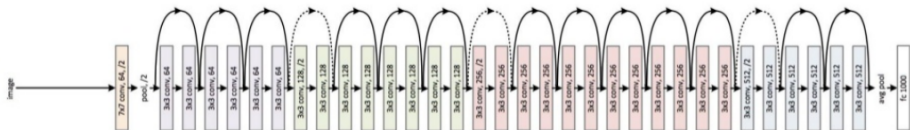
$$\begin{aligned} \frac{\delta E}{\delta x} &= \frac{\delta E}{\delta y} * \frac{\delta y}{\delta x} = \frac{\delta E}{\delta y} * (1 + F'(x)) \\ &= \frac{\delta E}{\delta y} + \frac{\delta E}{\delta y} * F'(x) \end{aligned}$$

Effet de la connexion résiduelle sur le gradient

- La connexion résiduelle ajoute un "court-circuit" pour le gradient
 - Le gradient de la sortie est rétropropagé directement à l'entrée : la norme est préservée quelque soit le nombre de couches !
- ⇒ diminue fortement les problèmes de gradients évanescents

Réseaux résiduels (ResNet)

- Empilement de blocs convolutifs similaire à VGG :
 - Convolutions 3×3
 - Dimensions spatiales $/2 \Rightarrow$ nombre de filtres $\times 2$



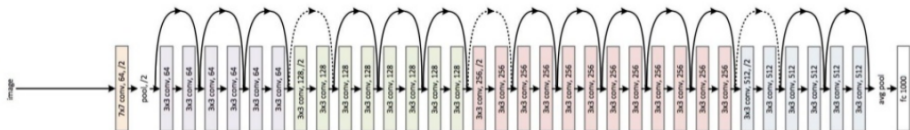
Architecture

- Pas de *maxpoolin*, convolution à pas de 2 pour certaines couches : ↘ dimensions spatiales, filtres $\times 2$
 $\Rightarrow$ préserve la complexité ($\sim$ nombre d'activations) par couche
- Si les dimensions changent : convolutions 1×1 pré-connexion résiduelle (pointillés)
- Couche finale : pooling adaptatif (moyenne) sur tout le tenseur d'activations

Entraînement

- Appris depuis une initialisation aléatoire, augmentation de données
- Pas de Dropout mais Batch Normalization après chaque convolution

ILSVRC'15 : ResNet

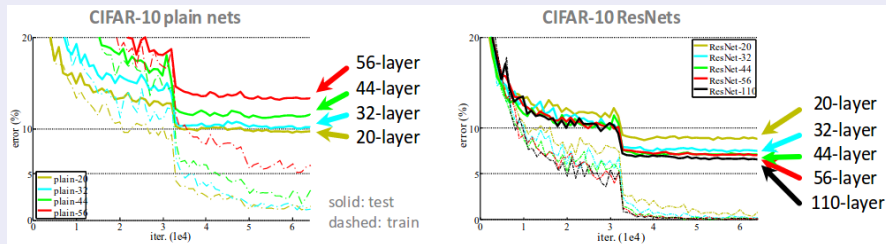


ResNet-18

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ResNet : résultats

Résultats sur CIFAR-10

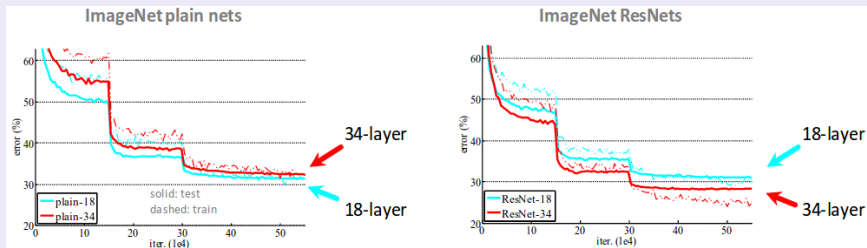


- Modèles résiduels profonds (*Deep ResNets*) : pas de difficulté particulière à l'apprentissage

⇒ modèles profonds ⇒ erreur d'apprentissage et de test plus faible

ResNet : résultats

Résultats sur ImageNet

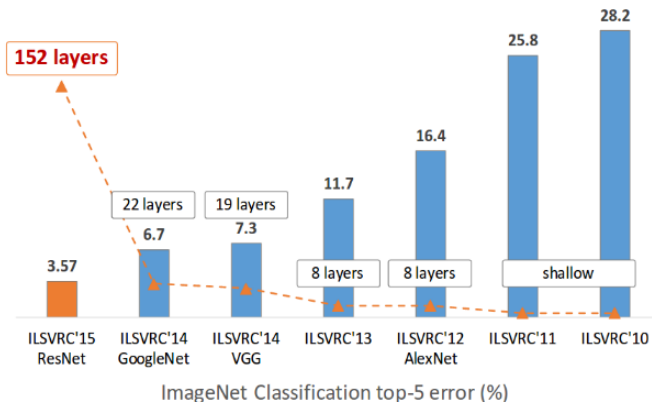


- Modèles résiduels profonds (*Deep ResNets*) : pas de difficulté particulière à l'apprentissage

⇒ modèles profonds ⇒ erreur d'apprentissage et de test plus faible

ResNet : conclusion

- Connexion résiduelle : moyen pour entraîner des réseaux plus profonds
- Solution à un problème d'optimisation, **pas** de capacité
- En pratique, **les erreurs sur ImageNet tombent sous la barre des 4%**



Plan du cours

1 Architectures résiduelles

2 Détection d'objet

3 Segmentation d'images

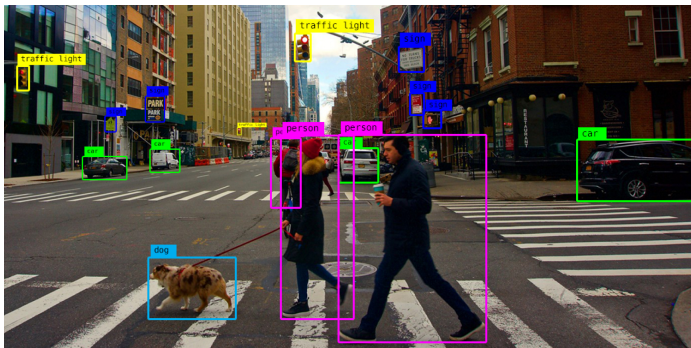
Détection d'objet

Définition

Localiser et identifier un objet dans une image.

$$\mathcal{I} \rightarrow C, (x, y)$$

- Entrée : image $\mathcal{I}$
- Sorties : classe C , position (x, y)
 - En pratique, la position est une **boîte englobante** (*bounding box*) : x, y, w, h



Évaluation de la détection d'objet

Intersection sur union



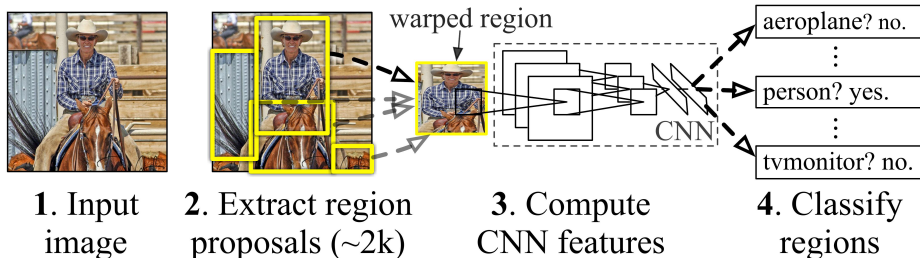
Métriques

Les détecteurs d'objet sont évalués à un seuil fixe $0 < \tau \leq 1$, typiquement $\tau = 0,5$.

- Vrai positif : objet prédit et $\text{IoU} > \tau$
- Faux positif : objet prédit et $\text{IoU} < \tau$
- Faux négatif : objet existant et $\text{IoU} < \tau$
- Précision (pour 1 classe) : $\frac{\text{vrai positifs}}{\text{vrais positifs} + \text{faux positifs}}$
- *Average Precision* (AP) (pour 1 classe) : moyenne de la précision pour différents seuils τ d'IoU ($\sim$ intégrale de la courbe précision-rappel)
- *mAP* (*mean Average Precision*) : moyenne de l'AP pour toutes les classes

R-CNN : *Regions with CNN features* GIRSHICK et al. 2014

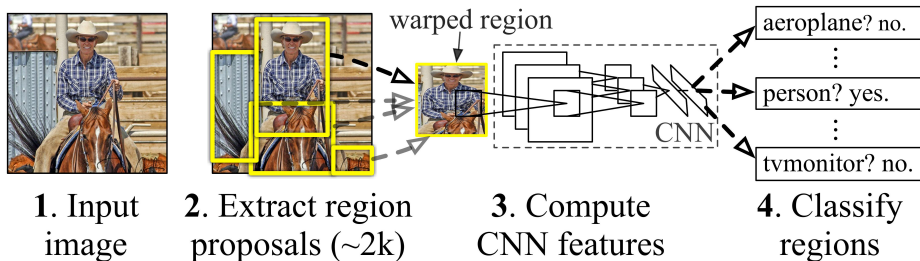
R-CNN: *Regions with CNN features*



- 1 Extraction des régions d'intérêt par *selective search* (algorithme non-deep)
- 2 Redimensionnement des imagerie en image 224×224 , calcul des *deep features*
- 3 Classification de l'imagerie en catégorie (SVM linéaire)

R-CNN : *Regions with CNN features* GIRSHICK et al. 2014

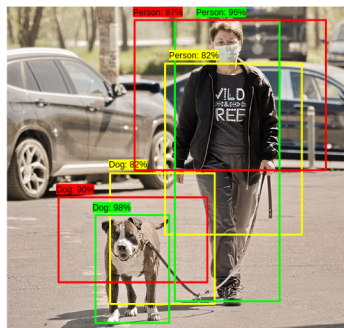
R-CNN: *Regions with CNN features*



Limitations

- Selective search renvoie de nombreuses régions qui se recouvrent : redondance du calcul des *deep features*
- Beaucoup de régions d'intérêt ne contiennent pas d'objet $\implies$ faux positifs lents à éliminer
- SVM apprise a posteriori : pas de *fine-tuning* des caractéristiques profondes

Post-traitement : Non-Maximum Suppression



Limiter les détections multiples

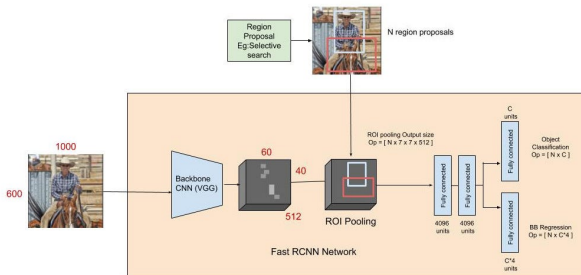
Inconvénient des détecteurs : des régions proches peuvent être classées plusieurs fois comme étant le même objet.

Solution : *non-maximum suppression*

Pour une classe C , parcourir chaque boîte englobante $\mathcal{B}$ prédite par ordre décroissant de son score de classe c :

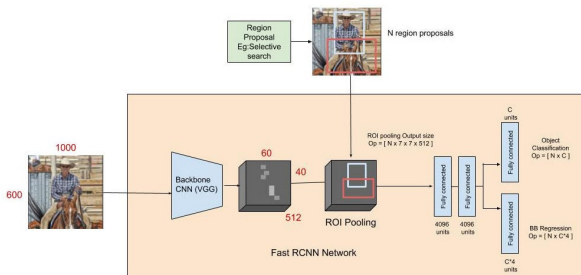
- 1 Parcourir toutes les boîtes englobante $\mathcal{B}'$ qui intersectent $\mathcal{B}$ qui ont la même classe,
- 2 Si $\text{IoU}(\mathcal{B}, \mathcal{B}') > \epsilon$, on supprime $\mathcal{B}'$.

Fast R-CNN GIRSHICK 2015



- 1 Calcul des caractéristiques profondes en utilisant seulement les couches convolutives d'un CNN
- 2 Extraction des régions d'intérêt par *selective search* (algorithme non-deep)
- 3 *Region of interest pooling* (RoI-pooling) : calculer la caractéristique moyenne sur chaque région
- 4 Classification de l'imagette en catégorie (SVM linéaire)
- 5 Régression des coordonnées de la boîte englobante (SVM linéaire) pour raffiner la proposition de *selective search*

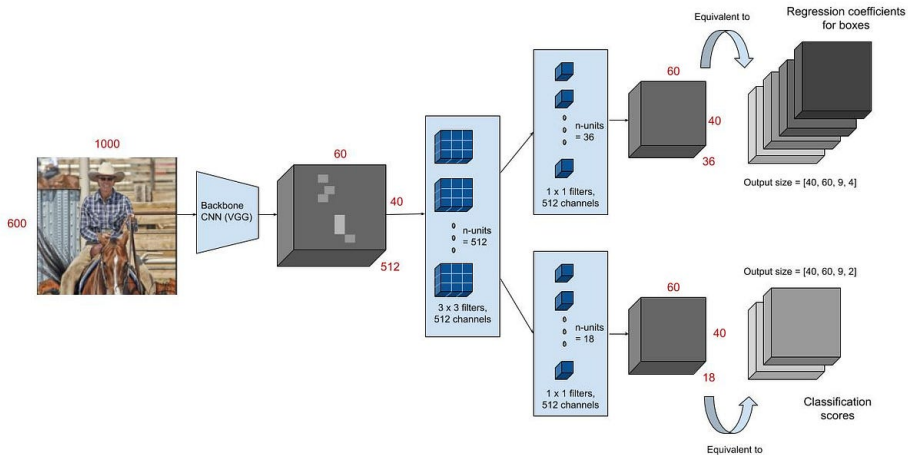
Fast R-CNN GIRSHICK 2015



- 1 Calcul des caractéristiques profondes en utilisant seulement les couches convolutives d'un CNN
- 2 Extraction des régions d'intérêt par *selective search* (algorithme non-deep)
- 3 *Region of interest pooling* (RoI-pooling) : calculer la caractéristique moyenne sur chaque région
- 4 Classification de l'imagette en catégorie (SVM linéaire)
- 5 Régression des coordonnées de la boîte englobante (SVM linéaire) pour raffiner la proposition de *selective search*

Avantage : plus besoin de recalculer les *features* pour chaque région !

Faster R-CNN REN et al. 2017



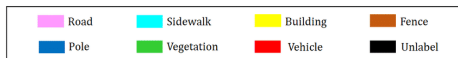
Plan du cours

1 Architectures résiduelles

2 Détection d'objet

3 Segmentation d'images

Segmentation sémantique



Définition

Produire une classification d'une image **pour chaque pixel**.

$$\mathcal{I} \rightarrow \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1w} \\ \vdots & \vdots & \vdots & \vdots \\ c_{h1} & c_{h2} & \cdots & c_{hw} \end{pmatrix}$$

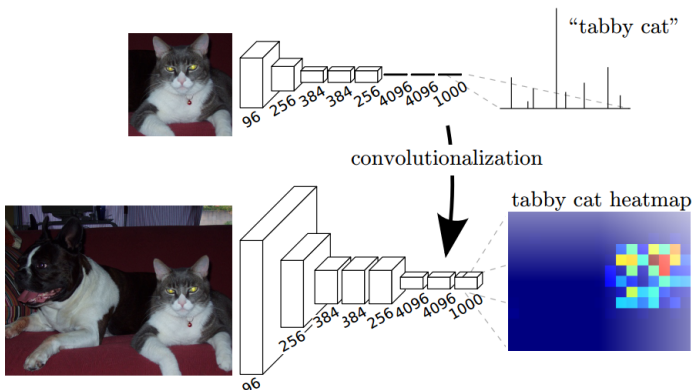
- Entrée : image $\mathcal{I} \in \mathbb{R}^{w \times h \times 3}$
- Sortie : carte sémantique $1, \dots, K^{w' \times h'}$ (K classes)

Métrique

- *mean IoU* (mIoU) : moyenne des IoU calculées pour chaque classe
- Fonction de coût : entropie croisée calculée pour chaque position

$$\mathcal{L} = \sum_{i=1}^{h'} \sum_{j=1}^{w'} \mathcal{L}_{\text{CE}}(y_{i,j}, y_{i,j}^*)$$

Fully convolutional networks LONG, SHELHAMER et DARRELL 2015

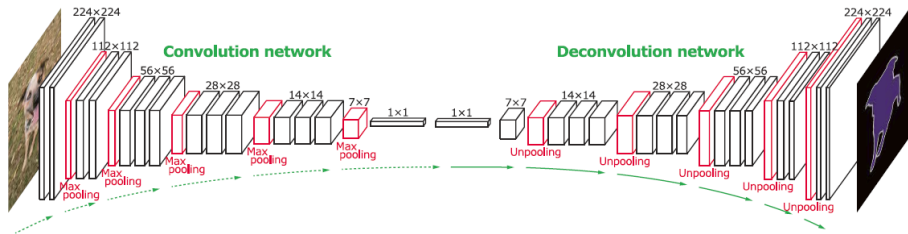


Convolutionalisation

Remplacement des couches entièrement connectées par une convolution 1×1 : permet de réaliser la classification pour chaque emplacement (i, j) du tenseur d'activation !

⇒ production d'une carte sémantique de dimensions (w', h') égales aux dimensions spatiales du tenseur en sortie de couche convolutive

DeconvNet NOH, HONG et HAN 2015

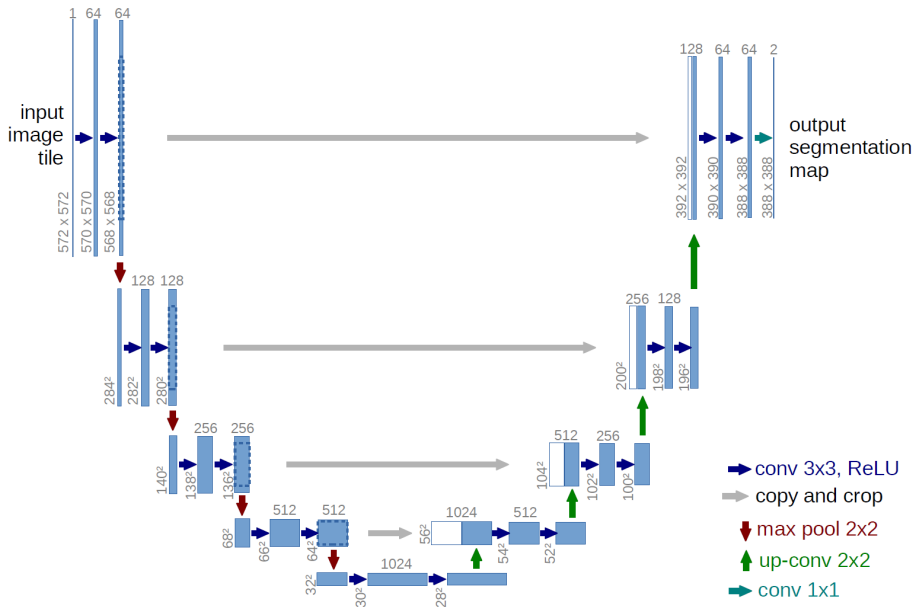


Encodeur-décodeur pour la segmentation

- Encodeur : couches convolutives classiques (par ex., VGG-16)
- Décodeur : symétrique en remplaçant les couches de *pooling* par du *unpooling*

Exemples d'opérations d'*unpooling* : *upsampling* (bilinéaire, plus proche voisin), déconvolution, *un-maxpooling*.

U-Net RONNEBERGER, FISCHER et BROX 2015



Bibliographie I

-  GIRSHICK, Ross (déc. 2015). "Fast R-CNN". In : *2015 IEEE International Conference on Computer Vision (ICCV)*. 2015 IEEE International Conference on Computer Vision (ICCV), p. 1440-1448. DOI : 10.1109/ICCV.2015.169.
-  GIRSHICK, Ross et al. (juin 2014). "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation". In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 580-587. DOI : 10.1109/CVPR.2014.81.
-  LONG, Jonathan, Evan SHELHAMER et Trevor DARRELL (juin 2015). "Fully Convolutional Networks for Semantic Segmentation". In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 3431-3440. DOI : 10.1109/CVPR.2015.7298965.
-  NOH, Hyeonwoo, Seunghoon HONG et Bohyung HAN (déc. 2015). "Learning Deconvolution Network for Semantic Segmentation". In : *2015 IEEE International Conference on Computer Vision (ICCV)*. 2015 IEEE International Conference on Computer Vision (ICCV), p. 1520-1528. DOI : 10.1109/ICCV.2015.178.
-  REN, Shaoqing et al. (juin 2017). "Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.6, p. 1137-1149. ISSN : 0162-8828. DOI : 10.1109/TPAMI.2016.2577031.

Bibliographie II



RONNEBERGER, Olaf, Philipp FISCHER et Thomas BROX (2015). "U-Net : Convolutional Networks for Biomedical Image Segmentation". In : *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015 : 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III*. Sous la dir. de Nassir NAVAB et al. Cham : Springer International Publishing, p. 234-241. ISBN : 978-3-319-24574-4. DOI : 10.1007/978-3-319-24574-4_28. URL : https://doi.org/10.1007/978-3-319-24574-4_28.